# Unit 2 Concept of OOPs

## INLINE FUNCTIONS

An inline function is a function that is expanded in line when it is invoked. Inline expansionmakes a program run faster because the overhead of a function call and return is eliminated. It is defined by using key word **"inline"**

**Necessity of Inline Function:**

• One of the objectives of using functions in a program is to save some memory space, which becomes appreciable when a function is likely to be called many times.

• Every time a function is called, it takes a lot of extra time in executing a series of instructions for tasks such as jumping to the function, saving registers, pushing arguments into the stack, and returning to the call ingfunction.

• When a function is small, a substantial percentage of execution time may be spent in such overheads.

• One solution to this problem is to use macro definitions, known as macros. Preprocessor macros arepopular in C. The major drawback with macros is that they are not really functions andtherefore, the usual error checking does not occur during compilation.

• C++ has different solution to this problem. To eliminate the cost of calls to small functions,

C++proposes a new feature called inline function.

General Form:

inline function-header

```
{
    function body;
}
```

Eg:

```
#include<iostream.h>
inline float mul(float x, float y)
        {
           return (x*y);
        }
inline double div(double p, double q)
        {
           return (p/q);
        }
   int main()
{
        float a=12.345;
        float b=9.82;
        cout<<mul(a,b);
        cout<<div(a,b);
     return 0;
  }
```

**Properties of inline function:**

1.Inline function sends request but not a command to compiler

2.Compiler my serve or ignore the request

3.if function has too many lines of code or if it has complicated logic then it is executed as normal function

**Situations where inline does not work:**

• A function that is returning value , if it contains switch ,loop or both then it is treated as normal function.

• if a function is not returning any value and it contains a return statement then it is treated as normal function

• If function contains static variables then it is executed as normal function

• If the inline function is declared as recursive function then it is executed as normal function.

## Function Overloading in C++

Function overloading is a feature of object-oriented programming where two or more functions can have the same name but different parameters. When a function name is overloaded with different jobs it is calledFunction Overloading. In Function Overloading "Function" name should be the same and the arguments should be different. Function overloading can be considered as an example of a polymorphism feature in C++.

The parameters should follow any one or more than one of the following conditions for Function overloading:

1. **Parameters should have a different type**

*add(int a, int b)*

*add(double a, double b)*

```
#include <iostream>

using namespace std;

void add(int a, int b)

{

   cout << "sum = " << (a + b);

}

void add(double a, double b)
```

```
{
cout << endl << "sum = " << (a + b);
}
// Driver code
int main()
{
add(10, 2);
add(5.3, 6.2);
return 0;
}
```

**2. Parameters should have a different number**

*add(int a, int b)*

*add(int a, int b, int c)*

```
#include <iostream>
using namespace std;
void add(int a, int b)
{
    cout << "sum = " << (a + b);
}
void add(int a, int b, int c){
cout << endl << "sum = " << (a + b + c);
}
// Driver code
int main()
```

```
{
add(10, 2);
add(5, 6, 4);
return 0;
}
```

**3. Parameters should have a different sequence of parameters.**

*add(int a, double b)*

*add(double a, int b)*

```
#include<iostream>
using namespace std;
void add(int a, double b)
{
cout<<"sum = "<<(a+b);
}
void add(double a, int b)
{
cout<<endl<<"sum = "<<(a+b);
}
// Driver code
int main()
{
add(10,2.5);
add(5.5,6);
return 0;
```

```
}
```

## Passing by Reference:

It allows a function to modify a variable without having to create a copy of it. We have to declare reference variables. The memory location of the passed variable and parameter is the same and therefore, any change to the parameter reflects in the variable as well.

```
#include <iostream>

using namespace std;

void swap(int& x, int& y)

{

int z = x;

x = y;

y = z;}

int main()

{

int a = 45, b = 35;

cout << "Before Swap\n";

cout << "a = " << a << " b = " << b << "\n";

swap(a, b);

cout << "After Swap with pass by reference\n";

cout << "a = " << a << " b = " << b << "\n";

}
```

## Memory Allocation for Objects:

Memory for objects is allocated when they are declared but not when class is defined. All objects in a given class use same member functions. The member functions are created and placed in memory only once when they are defined in class definition.

C++ supports these functions and also has two operators **new** and **delete**, that perform the task of allocating and freeing the memory in a better and easier way.

## new operator

The new operator denotes a request for memory allocation on the Free Store. If sufficient memory is available, a new operator initializes the memory and returns the address of the newly allocated and initialized memory to the pointer variable.

### Syntax to use new operator

pointer-variable = ***new*** data-type;

Here, pointer-variable is the pointer of type data-type. Data-type could be any built-in data type including array

or any user-defined data type including structure and class. ***Example:***

// Pointer initialized with NULL

// Then request memory for the variable

int *p = NULL;

p = new int;

## Initialize memory:

We can also initialize the memory for built-in data types using a new operator. For custom data types, a constructor is required (with the data type as input) for initializing the value. Here's an example of the initialization of both data types :

pointer-variable = ***new*** data-type(value);

**Example:**

*int *p = new int(25);*

*float *q = new float(75.25);*

**Allocate a block of memory:** new operator is also used to allocate a block(an array) of memory of

type *data-type*.

pointer-variable = **new** data-type[size];

where size(a variable) specifies the number of elements in an array.

**Example:**

int *p = new int[10]

## Delete operator

Since it is the programmer's responsibility to deallocate dynamically allocated memory, programmers are provided delete operator in C++ language.

**Syntax:**

// Release memory pointed by pointer-variable

**delete** pointer-variable;

Here, pointer-variable is the pointer that points to the data object created by *new*.

**Example:**

// It will free the entire array

// pointed by p.

delete[] p;

```
delete p;

delete q;
```

## FRIEND FUNCTIONS:

The private members cannot be accessed from outside the class. i.e. ... a non member function cannot have an access to the private data of a class. In C++ a non member function can access private by making the function friendly to a class.

A friend function is a function which is declared within a class and is defined outside the class. It does not require any scope resolution operator for defining. It can access private members of a class. It is declared by using keyword “friend”.

Ex:

```
class sample

{

int x,y;

public:

        sample(int a,int b);

        friend int sum(sample s);

        };sample::sample(int a,int b)

{

        x=a;y=b;

}

 int sum(samples s)

{ int sum;

   sum=s.x+s.y;

 return 0;

}
```

```
void main()
{
Sample obj(2,3);
  int res=sum(obj);
  cout<< “sum=”<<res<<endl;
}
```

A **friend function** possesses certain special characteristics:

• It is not in the scope of the class to which it has been declared as friend.

• Since it is not in the scope of the class, it cannot be called using the object of that class. It can be invoked like a normal function without the help of any object.

• Unlike member functions, it cannot access the member names directly and has to use an object name and dot membership operator with each member name.

• It can be declared either in the public or private part of a class without affecting its meaning.

• Usually, it has the objects as arguments.

```
#include<iostream.h>
class sample
{
  int a;int b;
  public:
    void setvalue()
  {
  a=25;
  b=40;
  }
```

```
friend float mean(sample s);
};
float mean(sample s)
{
  return float(s.a+s.b)/2.0;
}
int main()
{
  sample X;
  X.setvalue();
  cout<<"Mean value="<<mean(X);
  return 0;
}
```

**Write a program to find max of two numbers using friend function for two different classes**

```
#include<iostream>
using namespace std;
class sample2;
class sample1
{
int x;public:
sample1(int a);
friend void max(sample1 s1,sample2 s2)
};
sample1::sample1(int a)
```

```
{
 x=a;
}
 class sample2
{
 int y;
 public:
 sample2(int b);
 friend void max(sample1 s1,sample2 s2)
 };
 Sample2::sample2(int b)
{
 y=b;
}
   void max(sample1 s1,sample2 s2)
{
   If(s1.x>s2.y)
   cout<<”Data member in Object of class sample1 is larger ”<<endl;
   else
     cout<<”Data member in Object of class sample2 is larger ”<<endl;
     }
   void main()
{
 sample1 obj1(3);
 sample2 obj2(5);
```

```
  max(obj1,obj2);
}
```

**Write a program to add complex numbers using friend function**

```
#include<iostream>
using namespace std;
class complex
{
    float real,img;public:
    Complex();
    complex(float x,float y)
    friend complex add_complex(complex c1,complex c2);
  };
    complex::complex()
{
    real=img=0;
}
    complex::complex(float x,float y)
{
    real=x;img=y;
}
    complex add_complex(complex c1,complex c2)
{
    complex t;
t.real=c1.real+c2.real;
t.img=c1.img+c2.img;
```

```
return t;
}
void complex::display ()
{
   if(img<0)
   {img=-img;
   cout<<real<<"-i"<<img<<endl
   }else
       {
        cout<<real<<"+i"<<img<<endl
        }
       }
int main()
  {
       complex obj1(2,3);
       complex obj2(-4,-6);
       complex obj3=add_compex(obj1,obj2);
       obj3.display();
       return 0;
       }
```

## Friend Class:

A class can also be declared to be the friend of some other class. When we create a friend class then all the member functions of the friend class also become the friend of the other class. This requires the condition that the friend becoming class must be first declared or defined (forward declaration).

```
#include <iostream.h>

class sample_1

   {

   friend class sample_2;//declaring friend class

   int a,b;

   public:

   void getdata_1()

     {

        cout<<"Enter A & B values in class sample_1";

        cin>>a>>b;}

        void display_1()

           {

           cout<<"A="<<a<<endl;

           cout<<"B="<<b<<endl;

         }

};

class sample_2

  {

     int c,d,sum;

     sample_1 obj1;
```

```
    public:
      void getdata_2()
      {
    obj1.getdata_1();
      cout<<"Enter C & D values in class sample_2";
      cin>>c>>d;
        }
      void sum_2()
      {
    sum=obj1.a+obj1.b+c+d;
      }
    void display_2()
      {
        cout<<"A="<<obj1.a<<endl;cout<<"B="<<obj1.b<<endl;
        cout<<"C="<<c<<endl;
        cout<<"D="<<d<<endl;
        cout<<"SUM="<<sum<<endl;
          }
        };
int main()
{
   sample_1 s1;
   s1.getdata_1();
   s1.display_1();
   sample_2 s2;
```

```
    s2.getdata_2();
    s2.sum_2();
    s2.display_2();
}
```

Output:

```
    Enter A & B values in class sample_1:1 2
    A=1
    B=2
    Enter A & B values in class sample_1:1 2 3 4
    Enter C & D values in class sample_2:A=1
    B=2
    C=3
    D=4
SUM=10
```

# Function with Default Arguments (C++)

A function with default arguments is a function where one or more parameters are given default values.

If the caller does not pass those arguments, the default values are automatically used.

Syntax

```
return_type function_name(type parameter = default_value) {
    // function body
}
```

Example

```
#include <iostream>
using namespace std
void display(int x, int y = 10) {   // y has default value
    cout << "x = " << x << ", y = " << y << endl;
}
int main() {
    display(5);        // y = 10 (default)
    display(5, 20);    // y = 20 (user given)
    return 0;
}
```

**Output**

```
x = 5, y = 10
x = 5, y = 20
```

**Important Rules**

Default arguments are given from right to left.

```
void fun(int a, int b = 5, int c = 10);  // ✓ valid
```

Once a parameter has a default value, all parameters to its right must also have defaults.

```
void fun(int a = 5, int b);  // ✗ invalid
```

Default values are defined only once (usually in function declaration).

## Advantages

- Reduces function overloading
- Makes function calls simpler
- Improves code readability

# Constructors:

C++ provides a special member function called the constructor which enables an object to initialize itself when it is created.

A constructor is a special member function whose task is to initialize the objects of its class. It is special because its name is the same name as the class name. The constructor is invoked whenever an object of its associated class is created. It is called constructor because it constructs the values of data members of the class. A constructor is declared and defined as follows:

```
class integer

{

int m,n;

public:

integer( );

.....…

.....

};

integer :: integer( )

{

m=0;

n=0;

}

int main()

{ integer obj1;

 .....…

.....}
```

integer obj1; => not only creates object obj1 but also initializes its data members m and n to zero. There is no need to write any statement to invoke the constructor function.

# CHARACTERISTICS OF CONSTRUCTOR

• They should be declared in the public section.

• They are invoked automatically when the objects are created.

• They do not have return type, not even void.

• They cannot be inherited, though a derived class can call the base class constructor.

• Like other c++ functions, they can have default arguments.

• Constructors cannot be virtual.

• We cannot refer to their addresses.

• They make "implicit calls" to the operators new and delete when memory allocation is required.

Constructors are of 3 types:

1. Default Constructor

2. Parameterized Constructor

3. Copy Constructor

## 1. Default Constructor:

A constructor that accepts no parameters is called the default constructor.

```
#include<iostream.h>

#include<conio.h>

class item

{

 int m,n;

 public:
```

```
 item()
 {
  m=10;n=20;
 }
    void put();
};
      void item::put()
 {
  cout<<m<<n;
 }
 void main()
  {
  item t;
  t.put();
  getch();
 }
```

## 2. Parameterized Constructors:

The constructors that take parameters are called parameterized constructors.

```
#include<iostream.h>
class item
{
   int m,n;
   public:
   item(int x, int y)
```

```
{
m=x;
n=y;
}};
```

When a constructor has been parameterized, the object declaration statement such as item t; may not work. We must pass the initial values as arguments to the constructor function when an object is declared. This can be done in 2 ways:

item t=item(10,20); //***explicit call***

item t(10,20); //***implicit call***

Eg:

```
#include<iostream.h>
#include<conio.h>
 class item
   {
   int m,n;
   public:
   item(int x,int y)
   {
   m=x;
   n=y;
   }
   void put();
   };
 void item::put()
```

```
{
   cout<<m<<n;
}
   void main()
{
     item t1(10,20);
     item t2=item(20,30);
     t1.put();
     t2.put();
     getch();
}
```

## 3. Copy Constructor:

A copy constructor is used to declare and initialize an object from another object.

Eg:

item t2(t1);

or

item t2=t1;

1. The process of initializing through a copy constructor is known as copy initialization.

2. t2=t1 will not invoke copy constructor. t1 and t2 are objects, assigns the values of t1 to t2.

3. A copy constructor takes a reference to an object of the same class as itself as an argument.

```
#include<iostream.h>
```

```
class sample
{
 int n;
   public:
   sample()
   {
      n=0;
    }sample(int a)
      {
            n=a;
        }
     sample(sample &x)
{
           n=x.n;
}
   void display()
{
     cout<<n;
 }
};
     void main()
{
   sample A(100);
   sample B(A);
   sample C=A;
```

```
    sample D;
    D=A;
       A.display();
       B.display();
       C.display();
       D.display();
    }                                Output: 100 100 100 100
```

## Multiple Constructors in a Class:

Multiple constructors can be declared in a class. There can be any number of constructors in a class.

```
class complex
{
   float real,img;
   public:
   complex()//default constructor
{
     real=img=0;
}
complex(float r)//single parameter parameterized constructor
```

```
{
    real=img=r;
}
complex(float r,float i) //two parameter parameterized
constructor
{
    real=r;img=i;
}
complex(complex&c)//copy constructor
{
    real=c.real;
    img=c.img;
}complex sum(complex c )
{
   complex t;
   t.real=real+c.real;
   t.img=img+c.img;
   return t;
}
void show()
   {
    If(img>0)
    cout<<real<<"+i"<<img<<endl;
    else
```

```
    {
    img=-img;
    cout<<real<<"-i"<<img<<endl;
    }
   }
};
void main()
 {
   complex c1(1,2);
   complex c2(2,2);
   compex c3;
    c3=c1.sum(c3);
   c3.show();
}
```

# DESTRUCTORS:

A destructor, is used to destroy the objects that have been created by a constructor. Like a constructor, the destructor is a member function whose name is the same as the class name but is preceded by a tilde.

Eg: ~item() { }

1. A destructor never takes any argument nor does it return any value.

2. It will be invoked implicitly by the compiler upon exit from the program to clean up storage that is no longer accessible.

3. It is a good practice to declare destructors in a program since it releases memory space for future use.

```
 #include<iostream>
 using namespace std;
```

```
 class Marks
      {
       public:
      int maths;
      Int science;
       //constructor
      Marks() {
      cout << "Inside Constructor"<<endl;
      cout << "C++ Object created"<<endl;
      }
//Destructor
    ~Marks() {
      cout << "Inside Destructor"<<endl;
cout << "C++ Object destructed"<<endl;
}
       };
       int main( )
      {
      Marks m1;
      Marks m2;
      return 0;
      }
```

***Output:***

Inside Constructor

C++ Object created

Inside Constructor

C++ Object created

Inside Destructor

C++ Object destructed

Inside Destructor

C++ Object destructed

## This pointer:

To understand ‘this’ pointer, it is important to know how objects look at functions and data members of a class.

1. Each object gets its own copy of the data member.

2. All-access the same function definition as present in the code segment.

Meaning each object gets its own copy of data members and all objects share a single copy of member functions.

Then now question is that if only one copy of each member function exists and is used by multiple objects, how are the proper data members are accessed and updated?

The compiler supplies an implicit pointer along with the names of the functions as ‘this’. The ‘this’ pointer is passed as a hidden argument to all nonstatic member function calls and is available as a local variable within the body of all nonstatic functions. ‘this’ pointer is not available in static member functions as static member functions can be called without any object (with class name).

For a class X, the type of this pointer is ‘X* ‘. Also, if a member function of X is declared as const, then the type of this pointer is ‘const X *’

In the early version of C++ would let ‘this’ pointer to be changed; by doing so a programmer could change which object a method was working on. This feature was eventually removed, and now this in C++ is an r - value.

C++ lets object destroy themselves by calling the following code :`delete this;`

Following are the situations where ‘this’ pointer is used:

**1) When local variable’s name is same as member’s name**

```
#include<iostream>
using namespace std;
/* local variable is same as a member's name */
class Test
{
private:
int x;
public:
void setX (int x)
{
// The 'this' pointer is used to retrieve the object's x
// hidden by the local variable 'x'
this->x = x;
}
void print() { cout << "x = " << x << endl; }
};
int main()
{
Test obj;
int x = 20;
obj.setX(x);
obj.print();
return 0;
```

}

**2) To return reference to the calling object**

```
Test& Test::func ()
{
// Some processing
return *this;
}
```

When a reference to a local object is returned, the returned reference can be used to **chain function calls** on a single object.

```
#include<iostream>
using namespace std;
class Test
{
private:
int x;
int y;
public:Test(int x = 0, int y = 0) { this->x = x; this->y = y; }
Test &setX(int a) { x = a; return *this; }
Test &setY(int b) { y = b; return *this; }
void print() { cout << "x = " << x << " y = " << y << endl; }
};
int main()
{
Test obj1(5, 5);
```

```
// Chained function calls. All calls
```